

Beat by Bit

Documentação Técnica — Semestre 2

Resumo das mudanças

Migração da lógica do jogo do Python para a FPGA (Verilog).

O Python passou a funcionar exclusivamente como *renderer*, recebendo o estado do jogo via UART em pacotes de 20 bytes a 60 fps.

Projeto anterior:	projeto_sem1/
Projeto atual:	projeto_sem2/
FPGA:	Cyclone V (Intel/Altera)
Clock:	50 MHz (assumido)
UART:	115200 baud, 8N1
Simulador:	ModelSim / Icarus Verilog
Renderer:	Python 3 + Pygame

0 Sumário

1	Visão Geral	3
1.1	Objetivo	3
1.2	Divisão de responsabilidades	3
2	Arquitetura do Sistema	3
2.1	Diagrama de blocos	4
2.2	Hierarquia de módulos	4
3	Protocolo UART	4
3.1	Configuração	4
3.2	Formato do pacote (20 bytes)	5
3.3	Codificação de posição das notas	5
3.4	Estados da FSM (byte 1 do pacote)	6
4	Módulos Verilog Novos	6
4.1	uart_tx.v — Transmissor UART	6
4.2	lfsr8.v — Gerador Pseudoaleatório	6
4.3	note_track.v — Gerenciador de Notas por Trilha	7
4.4	packet_sender.v — Montagem e Envio do Pacote	7
4.5	fluxo_dados.v — Lógica Completa do Jogo	8
4.6	unidade_controle.v — FSM Principal	9
5	Python — Renderer	9
5.1	Fluxo de dados	9
5.2	Parser do pacote	9
5.3	Sincronização de pacotes	10
6	Como Fazer Funcionar na Prática	10
6.1	Requisitos	10
6.2	Passo 1: Configurar o Projeto no Quartus	10
6.3	Passo 2: Ajustar o Clock	10
6.4	Passo 3: Atribuir os Pinos (Pin Planner)	11
6.5	Passo 4: Compilar e Gravar	11
6.6	Passo 5: Instalar Dependências Python	12
6.7	Passo 6: Executar o Renderer	12
6.8	Indicadores visuais de status	12
6.9	Solução de Problemas	13
7	Simulação e Testes	13
7.1	Testbenches disponíveis	13
7.2	Executar com Icarus Verilog	13
7.3	Executar com ModelSim	14
7.4	Visualizar formas de onda	14
8	Resumo das Mudanças em Relação ao Semestre 1	14
8.1	Problemas identificados no Semestre 1	14
8.2	Soluções implementadas	14

Referências

15

1 Visão Geral

1.1 Objetivo

O projeto **Beat by Bit** é um jogo de ritmo implementado em FPGA. No semestre 1, toda a lógica do jogo — geração de notas, detecção de acerto, pontuação, temporizadores e condições de vitória — estava implementada em Python. A FPGA apenas transmitia o estado dos botões físicos para o computador.

No semestre 2, a lógica foi **completamente migrada para o Verilog**, respeitando o requisito do trabalho: *a FPGA deve ser o cérebro do jogo*. O Python passou a ser um *renderer* puro — só recebe pacotes via UART e desenha a tela.

1.2 Divisão de responsabilidades

Funcionalidade	Sem. 1	Sem. 2
Geração de notas (spawn)	Python	FPGA
Posição e movimento das notas	Python	FPGA
Detecção de acerto (hit window)	Python	FPGA
Pontuação (score / combo)	Python	FPGA
Contador de erros (misses)	Python	FPGA
Timer de jogo (60s)	Python	FPGA
Countdown (3s)	Parcial	FPGA
Condição de vitória/derrota	Python	FPGA
Máquina de estados (FSM)	FPGA	FPGA
Debounce e detecção de borda	FPGA	FPGA
Renderização gráfica	Python	Python

Tabela 1: Comparação de responsabilidades entre os semestres.

2 Arquitetura do Sistema

2.1 Diagrama de blocos

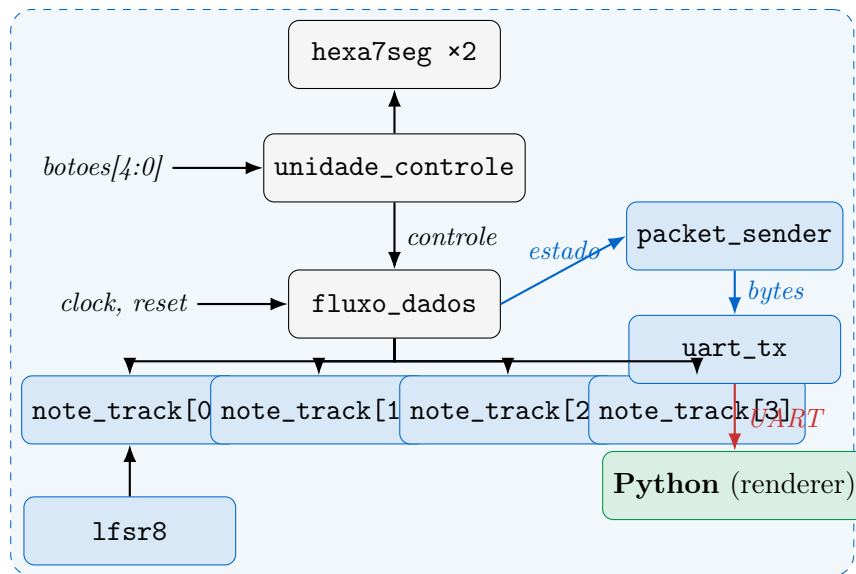


Figura 1: Diagrama de blocos do sistema. Módulos novos destacados em azul.

2.2 Hierarquia de módulos

Arquivo	Descrição	Status
projeto.v	Top-level; conecta todos os módulos	Atualizado
unidade_controle.v	FSM principal do jogo (6 estados)	Atualizado
fluxo_dados.v	Lógica completa do jogo	Reescrito
uart_tx.v	Transmissor UART 8N1	Novo
lfsr8.v	LFSR 8 bits para spawn aleatório	Novo
note_track.v	Gerenciador de notas por trilha	Novo
packet_sender.v	Montagem e envio do pacote UART	Novo
contador_m.v	Contador modulo-M parametrizável	Reutilizado
edge_detector.v	Detector de borda de subida	Reutilizado
registrador_4.v	Registrador 4 bits com enable	Reutilizado
hexa7seg.v	Decodificador para display 7 seg.	Reutilizado
interface.py	Renderer Python (Pygame)	Reescrito

Tabela 2: Hierarquia e status dos arquivos do projeto.

3 Protocolo UART

3.1 Configuração

- **Baud rate:** 115 200 bps
- **Formato:** 8N1 (8 bits de dados, sem paridade, 1 stop bit)
- **Direção:** FPGA → PC (unidirecional)

- **Taxa de envio:** 1 pacote por frame (≈ 60 fps)
- **Tempo por pacote:** $20 \times \frac{10}{115200} \approx 1,74$ ms

3.2 Formato do pacote (20 bytes)

Byte(s)	Nome	Descrição	Valores
0	HEADER	Marcador de início	0x55
1	state	Estado da FSM	0–5
2	countdown	Segundos restantes no countdown	3, 2, 1, 0
3	t0n0	Trilha 0 — age da nota 0	0–150, 0xFF
4	t0n1	Trilha 0 — age da nota 1	0–150, 0xFF
5	t0n2	Trilha 0 — age da nota 2	0–150, 0xFF
6–8	t1n0–t1n2	Trilha 1 (idem)	0–150, 0xFF
9–11	t2n0–t2n2	Trilha 2 (idem)	0–150, 0xFF
12–14	t3n0–t3n2	Trilha 3 (idem)	0–150, 0xFF
15	score_h	Pontuação (byte alto)	0–255
16	score_l	Pontuação (byte baixo)	0–255
17	misses	Contador de erros	0–10
18	combo	Combo atual	0–255
19	FOOTER	Marcador de fim	0xAA

Tabela 3: Estrutura do pacote UART de 20 bytes. 0xFF indica slot vazio.

3.3 Codificação de posição das notas

Cada nota é armazenada na FPGA como um contador de *age* (quadros desde o spawn). O Python converte em posição *Y* na tela pela fórmula:

$$Y = age \times NOTE_SPEED \quad (NOTE_SPEED = 4 \text{ px/frame}) \quad (1)$$

Age	Y (px)	Significado
0	0	Nota acabou de aparecer (topo da tela)
119	476	Borda inferior da hit zone
130	520	Centro da hit zone
141	564	Borda superior da hit zone
151	604	Nota saiu da tela → miss
0xFF	—	Slot vazio (sem nota)

Tabela 4: Mapeamento age → posição Y.

3.4 Estados da FSM (byte 1 do pacote)

Valor	Estado	Descrição
0	IDLE	Aguardando pressionar START
1	COUNTDOWN	Contagem regressiva 3-2-1
2	PLAY	Jogo em andamento
3	PAUSE	Jogo pausado
4	END_WIN	Vitória (60 s concluídos)
5	END_LOSE	Derrota (10 erros)

4 Módulos Verilog Novos

4.1 uart_tx.v — Transmissor UART

Função: Serializa um byte de 8 bits no formato 8N1 (1 start bit + 8 data bits + 1 stop bit).

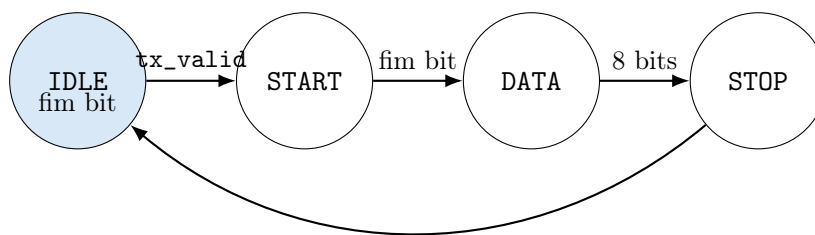
Parâmetro:

- $\text{CLKS_PER_BIT} = \lfloor f_{clk}/baud \rfloor = \lfloor 50\,000\,000/115\,200 \rfloor = 434$

Portas:

Sinal	Direção	Descrição
clock	entrada	Clock do sistema
reset	entrada	Reset assíncrono ativo alto
tx_data[7:0]	entrada	Byte a transmitir
tx_valid	entrada	Pulso de 1 ciclo para iniciar transmissão
tx_serial	saída	Sinal serial de saída (pino UART TX)
tx_ready	saída	1 quando pronto para novo byte

Diagrama de estados:



4.2 lfsr8.v — Gerador Pseudoaleatório

Função: LFSR (*Linear Feedback Shift Register*) de 8 bits para gerar spawn pseudoaleatório das notas.

Polinômio: $x^8 + x^6 + x^5 + x^4 + 1$ (taps nos bits 8, 6, 5, 4)

Propriedades:

- Período máximo de 255 estados (nunca retorna ao estado zero)
- Avança 1 passo a cada pulso de `enable` (`frame_tick`)

- Semente inicial: 0xAC

Uso no spawn: O `fluxo_dados` usa 2 bits do LFSR por trilha. Uma nota é spawnada se os 2 bits forem 00 (probabilidade $\approx 25\%$ por trilha por frame).

Listing 1: Lógica de feedback do LFSR8.

```
1 wire feedback = lfsr_reg[7] ^ lfsr_reg[5] ^ lfsr_reg[4] ^ lfsr_reg[3];
2 lfsr_reg <= {lfsr_reg[6:0], feedback}; // shift left com feedback
```

4.3 note_track.v — Gerenciador de Notas por Trilha

Função: Mantém até 3 notas simultâneas numa trilha. Cada nota é representada por um contador de age (quadros desde o spawn).

Constantes de jogo:

Constante	Valor	Significado
HIT_CENTER	130	Age quando a nota está no centro da hit zone ($Y = 520$)
HIT_WINDOW	11	Janela de tolerância ± 11 frames (± 44 px)
MISS_AGE	151	Age em que a nota sai da tela sem ser apertada
EMPTY	0xFF	Slot vazio

Lógica por frame_tick:

1. Para todos os slots válidos: `age++`
2. Se `age >= 151`: marca `escape_pulse=1`, libera slot
3. Se `spawn_en` e há slot livre: cria nota com `age=0`

Lógica por btn_press:

1. Procura nota com age dentro da janela [119, 141]
2. Se encontrada: `hit_pulse=1`, remove a nota
3. Caso contrário: `bad_press=1`

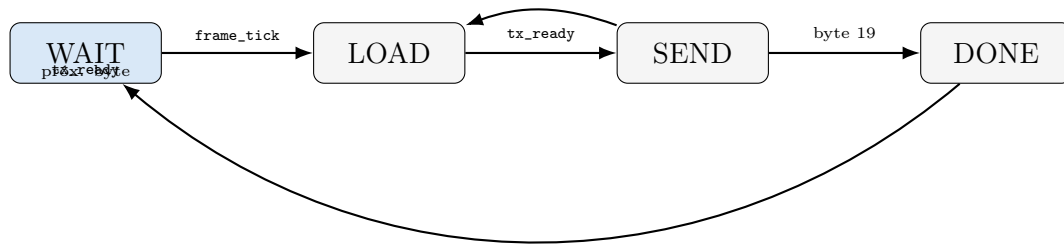
Saídas de pulso (duram exatamente 1 ciclo de clock):

Sinal	Significado
<code>hit_pulse</code>	Botão pressionado dentro da janela $\rightarrow$ acerto
<code>escape_pulse</code>	Nota passou sem ser apertada $\rightarrow$ miss
<code>bad_press</code>	Botão pressionado fora da janela $\rightarrow$ penalidade

4.4 packet_sender.v — Montagem e Envio do Pacote

Função: A cada `frame_tick`, captura o estado completo do jogo, monta o pacote de 20 bytes e envia byte a byte via `uart_tx`.

Máquina de estados interna:



O estado WAIT captura (*latch*) todos os sinais de entrada no momento do `frame_tick`, garantindo consistência do pacote. O envio acontece nos estados seguintes sem risco de leitura de dados em transição.

4.5 fluxo_dados.v — Lógica Completa do Jogo

Este módulo centraliza toda a lógica do jogo. Instancia os submódulos e mantém os registradores de estado.

Temporizadores:

Temporizador	M	N (bits)	Duração
Frame tick (<code>u_frame_counter</code>)	833 333	20	$\approx 1/60$ s
Countdown (<code>u_countdown</code>)	180	8	3 segundos (180 frames)
Timer de jogo (<code>u_game_timer</code>)	3 600	12	60 segundos

Tabela 5: Temporizadores baseados em `contador_m` instanciados no `fluxo_dados`. Clock de 50 MHz assumido.

Contadores de placar:

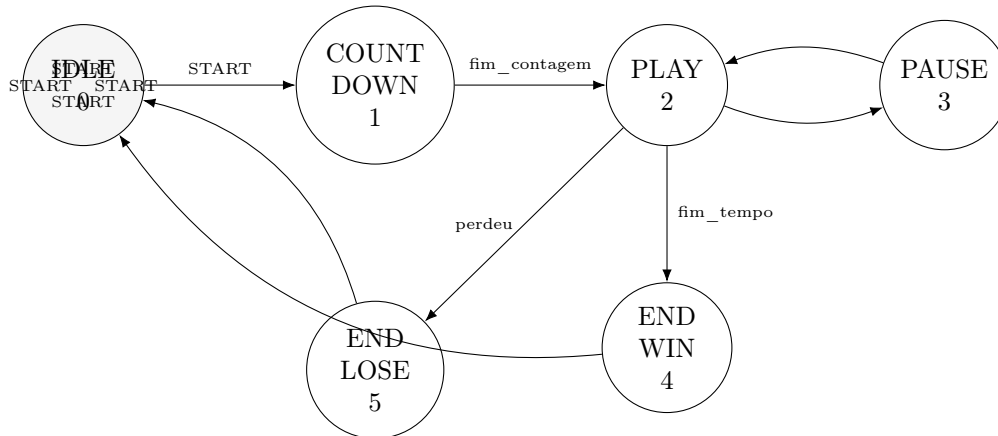
Listing 2: Lógica dos contadores de pontuação no `fluxo_dados.v`.

```

1 // Score (+100 acerto, -50 bad_press, minimo 0)
2 always @(posedge clock or posedge reset)
3     if (reset || limpaR) score_reg <= 0;
4     else if (any_hit) score_reg <= score_reg + 16'd100;
5     else if (any_bad) score_reg <= (score_reg >= 16'd50) ?
6         score_reg - 16'd50 : 16'd0;
7
8 // Misses (nota escape -> +1; perdeu quando >= 10)
9 always @(posedge clock or posedge reset)
10     if (reset || limpaR) miss_reg <= 0;
11     else if (any_escape && game_active) miss_reg <= miss_reg + 1;
12
13 assign perdeu = (miss_reg >= 8'd10);
14
15 // Combo (acerto -> +1; erro -> reset)
16 always @(posedge clock or posedge reset)
17     if (reset || limpaR) combo_reg <= 0;
18     else if (any_hit) combo_reg <= combo_reg + 1;
19     else if (any_bad || any_escape) combo_reg <= 0;
  
```

4.6 unidade_controle.v — FSM Principal

A FSM permaneceu com os mesmos 6 estados. A alteração foi a adição da saída `game_active` (HIGH somente no estado `PLAY`), que habilita o movimento das notas e os contadores de placar no `fluxo_dados`.



5 Python — Renderer

O `interface.py` foi reescrito para ser um *renderer* puro. Toda tomada de decisão de jogo foi removida.

5.1 Fluxo de dados

1. A thread `SerialReader` lê bytes da porta serial em background e os acumula em um buffer.
2. A cada frame, o loop principal chama `_process_serial()`, que drena o buffer e procura pacotes válidos usando `find_packet()`.
3. Ao encontrar um pacote com header `0x55` e footer `0xAA`, o `parse_packet()` extrai os campos.
4. O estado da FPGA é atualizado e a tela é renderizada de acordo.

5.2 Parser do pacote

Listing 3: Função `parse_packet` em `interface.py`.

```

1 def parse_packet(data, offset):
2     if data[offset] != 0x55 or data[offset + 19] != 0xAA:
3         return None
4     state      = data[offset + 1] & 0x0F
5     countdown  = data[offset + 2]
6     notes = []
7     for t in range(4):
8         track = [data[offset + 3 + t*3 + s] * NOTE_SPEED
9                 for s in range(3)]
10            if data[offset + 3 + t*3 + s] != 0xFF:
11                notes.append(track)
12    score = (data[offset + 15] << 8) | data[offset + 16]

```

```

13     misses = data[offset + 17]
14     combo  = data[offset + 18]
15     return {'state': state, 'countdown': countdown, 'notes': notes,
16           'score': score, 'misses': misses, 'combo': combo}

```

5.3 Sincronização de pacotes

A função `find_packet(buf)` varre o buffer em busca de um par 0x55 ... 0xAA separados por exatamente 18 bytes. Isso garante ressincronização automática em caso de perda de bytes na UART.

6 Como Fazer Funcionar na Prática

6.1 Requisitos

- **Hardware:** Placa FPGA com Cyclone V (ex: DE1-SoC, DE2-115)
- **Software FPGA:** Intel Quartus Prime (versão Lite é suficiente)
- **Software PC:** Python 3.8+, Pygame, pyserial
- **Cabo:** USB-A para mini/micro-USB (conexão UART da placa)

6.2 Passo 1: Configurar o Projeto no Quartus

1. Abra o Quartus Prime e crie um novo projeto ou abra o `beat_by_bit.qar` existente.
2. **Adicione os novos arquivos Verilog** ao projeto (*Project* → *Add/Remove Files*):
 - `uart_tx.v`
 - `lfsr8.v`
 - `note_track.v`
 - `packet_sender.v`
3. Certifique-se de que `projeto.v` está definido como **Top-Level Entity**.
4. Verifique se o dispositivo está configurado para o Cyclone V correto (*Assignments* → *Device*).

6.3 Passo 2: Ajustar o Clock

O parâmetro M do contador de frame é calculado por:

$$M_{frame} = \left\lfloor \frac{f_{clk}}{60} \right\rfloor \quad (2)$$

Clock da placa	M (frame tick)	N (bits)
50 MHz	833 333	20
100 MHz	1 666 667	21
25 MHz	416 667	19

Para verificar a frequência do clock da sua placa, consulte o manual ou o arquivo `c5_pin_model_dump.txt`. Edite `fluxo_dados.v` linha correspondente ao `u_frame_counter`:

```
1 contador_m #(M(833_333), .N(20)) u_frame_counter // 50 MHz
```

Da mesma forma, o parâmetro do `uart_tx` deve ser ajustado:

$$\text{CLKS_PER_BIT} = \left\lfloor \frac{f_{clk}}{115200} \right\rfloor \quad (3)$$

Edite `packet_sender.v`:

```
1 uart_tx #(.CLKS_PER_BIT(434)) u_tx // 50 MHz / 115200
```

6.4 Passo 3: Atribuir os Pinos (Pin Planner)

Abra o *Pin Planner* (*Assignments* → *Pin Planner*) e atribua:

Sinal no Verilog	Pino típico DE1-SoC	Descrição
clock	PIN_AF14	Clock de 50 MHz
reset	KEY[0]	Botão de reset
botoes[0]	KEY[1]	Trilha amarela (D)
botoes[1]	SW[0] ou KEY	Trilha azul (F)
botoes[2]	SW[1] ou KEY	Trilha verde (J)
botoes[3]	SW[2] ou KEY	Trilha vermelha (K)
botoes[4]	KEY[3]	START / PAUSE
uart_tx_out	PIN do UART TX	TX da UART USB
db_estado	HEX0[6:0]	Display 7 segmentos
db_jogada	HEX1[6:0]	Display 7 segmentos
leds_pulsos	LEDR[4:0]	LEDs de debug

Tabela 6: Atribuição de pinos para placa DE1-SoC. Ajuste conforme o manual da sua placa.

Atenção — Pino UART TX: O pino de TX da UART USB varia conforme a placa. Na DE1-SoC é geralmente ligado ao *HPS UART*. Consulte o *User Manual* da placa e o arquivo `c5_pin_model_dump.txt` para localizar o pino correto. É possível também usar qualquer GPIO livre e conectar um adaptador USB-Serial externo.

6.5 Passo 4: Compilar e Gravar

```
# No Quartus: Processing -> Start Compilation
# Verificar: 0 erros críticos no Flow Summary

# Gravar na FPGA: Tools -> Programmer
# Selecionar arquivo .sof gerado
# Clicar em "Start"
```

6.6 Passo 5: Instalar Dependências Python

```
cd /home/zocal/labdig_files/projeto_sem2

# Ativar o ambiente virtual (se existir)
source env/bin/activate

# Instalar dependencias (caso necessario)
pip install pygame pyserial
```

6.7 Passo 6: Executar o Renderer

```
# Verificar a porta serial da FPGA
ls /dev/ttyUSB*      # Linux
# ou
ls /dev/cu.usbserial* # macOS

# Executar
python interface.py --port /dev/ttyUSB0 --baud 115200
```

Parâmetros de linha de comando:

Argumento	Padrão	Descrição
-port	/dev/ttyUSB0	Porta serial da FPGA
-baud	115200	Baud rate

6.8 Indicadores visuais de status

No canto superior direito da tela, o renderer exibe:

- **FPGA ON** — conexão UART estabelecida
- **FPGA OFF** — sem conexão (modo demo, tela fica na última posição recebida)

6.9 Solução de Problemas

Sintoma	Solução
Tela permanece em IDLE	Verificar se a FPGA está gravada e transmitindo. Testar com <code>minicom</code> ou <code>screen /dev/ttyUSB0 115200</code> .
Notas aparecem em posições erradas	Verificar se o clock da FPGA é 50 MHz. Se diferente, ajustar M no <code>fluxo_dados.v</code> .
Jogo muito lento ou rápido	O <code>NOTE_SPEED</code> no Python deve ser igual ao usado no cálculo Verilog (ambos = 4).
<code>/dev/ttyUSB0</code> não existe	Verificar driver USB-UART (<code>lsusb</code>), adicionar usuário ao grupo serial: <code>sudo usermod -aG dialout \$USER</code> .
Permissão negada na porta serial	Executar <code>sudo chmod 666 /dev/ttyUSB0</code> ou adicionar ao grupo <code>dialout</code> .
Pacotes corrompidos / estado errado	O <code>find_packet</code> se ressincroniza automaticamente. Se persistir, verificar conexão do cabo e gravar novamente na FPGA.

Tabela 7: Solução de problemas comuns.

7 Simulação e Testes

7.1 Testbenches disponíveis

Arquivo	Módulo testado	Cenários cobertos
<code>tb_uart_tx.v</code>	<code>uart_tx</code>	Envio de 0x55, 0xAA, 0xFF, 0x00; <code>tx_ready</code>
<code>tb_lfsr8.v</code>	<code>lfsr8</code>	Nunca zero, período=255, freeze sem enable
<code>tb_note_track.v</code>	<code>note_track</code>	Spawn, acerto (borda inf/centro/borda sup), escape, <code>bad_press</code> , 3 slots, overflow, <code>game_active=0</code>
<code>tb_packet_sender.v</code>	<code>packet_sender</code>	Todos os 20 bytes corretos, segundo pacote

7.2 Executar com Icarus Verilog

```
cd /home/zocal/labdig_files/projeto_sem2

# Todos os testbenches
./run_tb.sh

# Um testbench específico
./run_tb.sh tb_note_track
```

7.3 Executar com ModelSim

```
# Compilar
vlog uart_tx.v lfsr8.v note_track.v tb_note_track.v

# Simular
vsim tb_note_track

# No console do ModelSim:
run -all
```

7.4 Visualizar formas de onda

Os testbenches geram arquivos `.vcd` que podem ser abertos no GTKWave:

```
gtkwave tb_note_track.vcd &
```

Sinais recomendados para inspecionar em `tb_note_track.vcd`:

- `frame_tick` — pulsos de 60 fps
- `note0`, `note1`, `note2` — ages das notas
- `hit_pulse`, `bad_press`, `escape_pulse` — eventos de jogo
- `btn_press` — pressão do botão

8 Resumo das Mudanças em Relação ao Semestre 1

8.1 Problemas identificados no Semestre 1

1. `fim_tempo` e `perdeu` eram placeholders:

```
1 assign fim_tempo = botoes_raw[0] & botoes_raw[1]; // incorreto
2 assign perdeu   = botoes_raw[2] & botoes_raw[3]; // incorreto
```

Esses sinais não tinham lógica real — eram ativados acidentalmente ao pressionar dois botões simultâneos.

2. **Contagem de tempo no contador_m:** O parâmetro $M=3000$ para 3 segundos implicava clock de 1 kHz, provavelmente incorreto para a placa real.
3. **Toda a lógica de jogo no Python:** notas, acertos, score, timer de 60s e condição de derrota estavam implementados em Python, contradizendo o objetivo do trabalho.

8.2 Soluções implementadas

1. `fim_tempo` agora vem do `contador_m` com $M = 3600$ frames ($= 60$ s) no `fluxo_dados`.
2. `perdeu` agora é `miss_reg >= 10`, calculado em hardware.
3. `frame_tick` gerado com $M = 833\,333$ ciclos (50 MHz/60 fps).
4. Quatro módulos novos implementam a lógica do jogo completamente em Verilog.
5. Python reduzido a um renderer de ≈ 270 linhas (era 470 linhas com toda a lógica).

8 Referências

1. Intel/Altera. *Cyclone V Device Handbook*. Disponível em: <https://www.intel.com/content/www/us/en/programmable/documentation/sam1403481054977.html>
2. Terasic. *DE1-SoC User Manual*. Disponível em: <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&No=836&PartNo=4>
3. Pygame. *Pygame Documentation*. Disponível em: <https://www.pygame.org/docs/>
4. pySerial. *pySerial Documentation*. Disponível em: <https://pyserial.readthedocs.io/>
5. Icarus Verilog. *Icarus Verilog*. Disponível em: <http://iverilog.icarus.com/>